

Run your lab's AI access from your AI agent

One endpoint for Google's Gemini models. You add members, give each one a capped key, give an app its own key, and watch what the lab spends, by asking in plain language.

One lab allowance
Resets on the 1st. You split it into caps per member.

A key per person, or per app
By one-time claim link; each person signs in and accepts the terms.

Plans before changes
Keys and removals are two-step: a plan, then your yes.

Usage, not prompts
The gateway records who, model, tokens and cost. Never the text.

1

Plug aigw into your agent

5 min

aigw is an MCP server: any MCP-capable agent works. Research Computing adds you to the sign-in list; then (pick one):

HERMES

\$ hermes mcp add aigw --url https://ai-gateway-mcp-575977597413.us-central1.run.app/mcp --auth oauth

CLAUDE CODE

\$ claude mcp add --transport http aigw https://ai-gateway-mcp-575977597413.us-central1.run.app/mcp

OPENCODE

\$ opencode mcp add aigw --url https://ai-gateway-mcp-575977597413.us-central1.run.app/mcp

GEMINI CLI

\$ gemini mcp add -s user -t http aigw https://ai-gateway-mcp-575977597413.us-central1.run.app/mcp

A browser opens once: sign in with your UCR NetID account. Then ask: *"Who am I on the AI gateway?"*

✓ It shows your NetID, your lab, and you as its PI.

2

Add your students, each with a cap

1 min each

"Add jdoe001 to my lab with a monthly cap of 50 dollars."

Use NetIDs. The directory name comes back so you can confirm the person. Leave out the cap and the lab default applies.

3

Give each student a key

1 min each

"Issue a key to jdoe001." Confirm the plan, then email the claim link and message it returns.

The link holds no key, works once, only for that student, for 72 hours. Keys expire after 90 days, cap each day at 30% of the monthly cap, carry rate limits and the Gemini models.

✓ The student signs in, accepts the terms and sees their key once.

4

Watch and adjust

YOU SAY	AIGW DOES
"Show my lab."	allowance, spent, reset date, every member's cap and spend
"What did my lab spend in the last 30 days?"	by member, by model, by day
"Set jdoe001's cap to 100 dollars."	changes the cap (never above the allowance)
"Block jdoe001's key." / "Remove jdoe001."	refuses the key at once / takes access away, history kept
"Issue an app key named teaching-ai with a monthly cap of 1,000 dollars."	one key for an app many people use; PI only; open the link yourself, put the key in the app's settings
"Make jsmith002 a delegate."	a lab manager who can add members and issue personal keys
"Freeze my lab."	blocks every key in the lab until you unfreeze

Collect your key and start using it

1. Collect your key

Open the claim link from your PI, sign in with your **UCR NetID account**, accept the terms and click **Show my key**. It is shown once: copy it straight into an environment variable, not your code.

```
$ export UCR_AI_GATEWAY_KEY=your-key # add to
~/.zshrc or ~/.bashrc
```

2. Use it (OpenAI-compatible)

Base URL for Python, OpenCode and most tools:

```
https://ucr-ursa-major-ai-gateway-service-575977597413.us-central1.run.app/v1
```

PYTHON · PIP INSTALL OPENAI

```
from openai import OpenAI; import os
client = OpenAI(base_url="<base URL above>",
                 api_key=os.environ["UCR_AI_GATEWAY_KEY"])
r = client.chat.completions.create(model="gemini-3.8-fla
sh",
    messages=[{"role": "user", "content": "hello"}])
print(r.choices[0].message.content)
```

YOUR SPEND AND LIMITS (FREE)

```
$ curl https://ucr-ursa-major-ai-gateway-service-575977597413.us-central1.run.app/key/info -H "Authorization: Bearer $UCR_AI_GATEWAY_KEY"
```

Gemini CLI, OpenCode: exact setup lines on the web page (QR below).

3. Pick a model

MODEL	GOOD FOR
gemini-3.8-flash	fast, low cost; the sensible default
gemini-2.5-flash-lite	cheapest; bulk classification, extraction
gemini-2.5-pro	deeper reasoning, higher price
gemini-embedding-001	embeddings for search and retrieval

Model choice is the biggest cost lever. Flash costs a fraction of Pro.

Need another model? The gateway can route to other models and providers, and more are planned. Ask Research Computing.

Rules of the road

- **P1 and P2 data only.** No student records, health information, ID or account numbers, export-controlled data or CUI without written ISO approval.
- **Your key is yours.** Never in code, a repo, chat, email or a shared file. Never shared.
- **Leaked?** Report it at once: it is blocked immediately. Email research-computing@ucr.edu.
- **UCR work only,** under UC policy (Electronic Communications Policy, IS-3).
- **A pilot.** Allowances, models and terms can change.

If you get stuck

401 from the gateway	key wrong, blocked or expired; check <code>/key/info</code> , then ask your PI
error naming a model	your key does not include it; pick one from the table
"budget exceeded"	daily or monthly cap reached; it resets, or ask your PI
429 rate limit	too many requests a minute; slow down, or ask your PI
link "issued to someone else"	sign in with the NetID account the link names
link expired	links last 72 hours; ask your PI for a new one
lost your key	it is never shown again; ask your PI to issue a new one

What the gateway sees

Who, which model, tokens and cost, so your PI can see spend. **Not** the text of your prompts or the answers. Requests are processed by Google Cloud (Vertex AI) on UCR's account.



This sheet, online

Copyable commands for every client
ucr-research-computing.github.io/kb/ai-gateway-lab-guide

Help: research-computing@ucr.edu with your NetID, lab, time and error. Never your key.